

# ■ Keyboard Sprint Championship

---

Module: ECS414U — Object Oriented Programming

# PART I — Textual Typing Race Simulator

## Task 1: The Typist Class

The `Typist` class models a single competitor in the race. It stores identity, accuracy, progress, and burnout state — all as private fields, accessed only through validated public methods. This design is the foundation of encapsulation.

### 1.1 Fields

| Field                 | Type    | Purpose                                            |
|-----------------------|---------|----------------------------------------------------|
| name                  | String  | Display name of the typist                         |
| symbol                | char    | Unicode character shown on the race track          |
| progress              | int     | Characters correctly typed; always $\geq 0$        |
| accuracy              | double  | Typing accuracy in $[0.0, 1.0]$                    |
| burntOut              | boolean | True while frozen in a burnout episode             |
| burnoutTurnsRemaining | int     | Countdown turns until recovery; 0 if not burnt out |

### 1.2 Constructor

The constructor has exactly the required signature:

```
public Typist(char typistSymbol, String typistName, double typistAccuracy)
```

All six fields are initialised. `accuracy` is set via `setAccuracy()` so that out-of-range values passed at construction time are automatically clamped.

### 1.3 Methods — Accessors (Getters)

Accessors return field values without modifying state:

| Method                     | Returns | Notes                                     |
|----------------------------|---------|-------------------------------------------|
| getName()                  | String  | Typist's display name                     |
| getSymbol()                | char    | Unicode race-track symbol                 |
| getProgress()              | int     | Current character position                |
| getAccuracy()              | double  | Accuracy in $[0.0, 1.0]$                  |
| isBurntOut()               | boolean | True if currently frozen                  |
| getBurnoutTurnsRemaining() | int     | Turns left in burnout; 0 if not burnt out |

#### 1.4 Methods — Mutators (Setters) with Validation `slideBack(int amount)`

Moves the typist backwards by *amount* characters. The critical validation is a floor clamp: after subtracting, if `progress` falls below zero it is reset to zero. Without this clamp, a typist at position 1 sliding back 5 characters would reach position -4, which has no valid meaning on a forward-only track.

```
public void slideBack(int amount) {
    progress -= amount;
    if (progress < 0) { progress = 0; } }
```

#### `setAccuracy(double newAccuracy)`

Updates the `accuracy` field. Values outside `[0.0, 1.0]` are clamped: below 0.0 becomes 0.0; above 1.0 becomes 1.0. This is essential because `accuracy` is used as a probability in `Math.random()` comparisons — a value of -0.3 or 1.7 would break the simulation logic entirely.

```
public void setAccuracy(double newAccuracy) {
    if (newAccuracy < 0.0) accuracy = 0.0;
    else if (newAccuracy > 1.0) accuracy = 1.0;
    else accuracy = newAccuracy;
}
```

#### Other Mutators

| Method                            | Behaviour                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>typeCharacter()</code>      | Increments <code>progress</code> by 1 only if not burnt out                                                     |
| <code>burnOut(int turns)</code>   | Sets <code>burntOut=true</code> and <code>burnoutTurnsRemaining=turns</code> ; ignores <code>turns&lt;=0</code> |
| <code>recoverFromBurnout()</code> | Decrements counter; clears flag when counter reaches 0                                                          |
| <code>resetToStart()</code>       | Sets <code>progress=0</code> , <code>burntOut=false</code> , <code>burnoutTurnsRemaining=0</code>               |
| <code>setSymbol(char)</code>      | Replaces the track symbol                                                                                       |

#### 1.5 Encapsulation

All six fields are declared **private**. External code cannot read or modify them directly — every interaction goes through a method. This means the class can enforce invariants (`progress >= 0`, `accuracy` in `[0,1]`) regardless of what the calling code does. If `progress` were public, any class could set it to -100 and break the simulation.

The only methods that change state are the mutators listed above; all other public methods are pure accessors that simply return a field value.

#### 1.6 Testing Evidence

The following table summarises all tests run against the `Typist` class, what they verify, and the expected vs. actual outcome.

| Test                              | Input / Action                           | Expected                          | Actual                  | Pass ? |
|-----------------------------------|------------------------------------------|-----------------------------------|-------------------------|--------|
| <code>slideBack</code> at zero    | <code>progress=0, slideBack(5)</code>    | <code>progress</code> stays 0     | <code>progress=0</code> | YES    |
| <code>slideBack</code> partial    | <code>progress=3, slideBack(2)</code>    | <code>progress=1</code>           | <code>progress=1</code> | YES    |
| <code>slideBack</code> overshoot  | <code>progress=2, slideBack(10)</code>   | <code>progress=0</code> (clamped) | <code>progress=0</code> | YES    |
| <code>typeCharacter</code> normal | <code>progress=5, typeCharacter()</code> | <code>progress=6</code>           | <code>progress=6</code> | YES    |

|                               |                                |                              |               |     |
|-------------------------------|--------------------------------|------------------------------|---------------|-----|
| typeCharacter t<br>r burnt ou | burnOut(3), typeCharacter()    | progress unchanged           | unchanged     | YES |
| burnout<br>countdown          | burnOut(3), 3x<br>recoverFromB | isBurntOut()=false           | false after 3 | YES |
| resetToStart<br>clears all    | progress=10,burnOut(2),reset   | toStart()progress=0,notBurnt | cleared       | YES |
| accuracy<br>clamp low         | setAccuracy(-0.5)              | accuracy=0.0                 | accuracy=0.0  | YES |
| accuracy<br>clamp high        | setAccuracy(1.8)               | accuracy=1.0                 | accuracy=1.0  | YES |
| accuracy<br>valid             | setAccuracy(0.75)              | accuracy=0.75                | accuracy=0.75 | YES |

## Task 2: Improvements to the TypingRace Class

The original TypingRace class left by Ty Posaurus contained several bugs and design problems. All were identified, explained, and corrected in the new implementation.

### BUG 1 — Win condition used == instead of >=

**Problem:** The original raceFinishedBy() used t.getProgress() == passageLength. Because typeCharacter() increments by 1 per turn, progress can overshoot the end (e.g., a typist at position 39 on a length-40 track advances to 40 correctly, but if another advance happened in the same turn they could reach 41). Using == means position 41 never triggers the win condition — the race runs forever. **Fix:** Changed to t.getProgress() >= passageLength.

```
private boolean hasFinished(Typist t) {
    return t.getProgress() >= passageLength; }
```

### BUG 2 — Accuracy boost applied every remaining turn after the win

**Problem:** In the original, the winner was detected inside the while loop but the accuracy boost (winner.setAccuracy(...)) was also inside the loop. After the first win detection the loop continued for one extra turn, applying the boost again.

**Fix:** Moved the accuracy boost to after the while loop exits, applied exactly once.

```
winner.setAccuracy(winner.getAccuracy() + WIN_ACCURACY_BOOST); // after loop
```

### BUG 3 — Burnout rolled during an existing burnout turn

**Problem:** The original advanceTypist() called the burnout probability check unconditionally. If a typist was already burnt out, the burnout counter could be reset to 3 every turn, making burnout effectively permanent for high-accuracy typists.

**Fix:** Added an early return at the start of processTurn() when isBurntOut() is true.

```
if (t.isBurntOut()) {  
    t.recoverFromBurnout();    return; // skip typing entirely  
}
```

#### BUG 4 — Hardcoded seat1/seat2/seat3 fields limited typists to exactly 3

**Problem:** The original used three separate Typist fields. Adding a fourth typist required changing the class structure. The spec allows 2–6 typists in the GUI.

**Fix:** Replaced seat fields with a List. addTypist(Typist t) appends to the list.

```
private final List typists = new ArrayList<>();  
public void addTypist(Typist t) { typists.add(t); }
```

#### BUG 5 — Thread.sleep used wrong exception class

**Problem:** The original caught a bare Exception for Thread.sleep, swallowing the InterruptedException without re-interrupting the thread, which is poor practice and can cause thread-management issues in longer-running applications.

**Fix:** Catch InterruptedException specifically and call Thread.currentThread().interrupt().

```
} catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

## PART II — GUI Module Development

The graphical version is built entirely in Swing and organised as a three-tab application: Setup, Race, and Results. It shares no visual elements with any basic JProgressBar layout.

### Architecture Overview

| Component        | Class / Widget  | Purpose                                          |
|------------------|-----------------|--------------------------------------------------|
| Application root | TypingRaceGUI   | JFrame hosting a JTabbedPane with 3 tabs         |
| Setup tab        | JPanel (custom) | Passage chooser, seat count, per-typist config   |
| Race tab         | TrackCanvas     | Custom-painted animated race track (inner class) |
| Results tab      | JTable x2       | Per-race stats + cumulative leaderboard          |
| Simulation       | java.util.Timer | Fires a tick every 220 ms on the EDT             |

### Interactive Race Configuration (Requirement 1)

- Passage Selection: ComboBox with Short (25), Medium (50), Long (80), or Custom (JSpinner). The passage length drives the track width on the TrackCanvas.
- Seat Count: JSpinner clamped to [2, 6]. Changing it shows/hides per-seat configuration rows.
- Autocorrect: When checked, slideBack amount is halved from 2 to 1.
- Caffeine Mode: First 10 ticks give all typists +0.08 accuracy; ticks 11+ double burnout risk.
- Night Shift: Applies a -0.05 accuracy penalty to all typists at race launch.

### Customisable Typists (Requirement 2)

| Customisation   | Options                                      | Accuracy Effect                                     |
|-----------------|----------------------------------------------|-----------------------------------------------------|
| Typing Style    | Touch Typist / Hunt & Peck / Phone Thumbs    | / Voice-to-Text+0.05 / -0.10 / -0.05 / 0.00         |
| Keyboard Type   | Mechanical / Membrane / Touchscreen / Ste    | nographySpeed multiplier: 1.05 / 1.00 / 0.90 / 1.20 |
| Accessory       | None / Wrist Support / Energy Drink / Noise- | CancellingAccuracy +0 / +0.02 / +0.03 / +0.04       |
| Accuracy Slider | 0 – 100 (mapped to 0.0 – 1.0)                | Direct base accuracy                                |
| Symbol          | Any single character                         | Visual only                                         |

### Statistics and Analytics (Requirement 3)

- WPM: Calculated as (passageLength / 5) / (seconds elapsed) after each race.
- Accuracy %: Displayed from the Typist's final accuracy field.

- Burnout Count: Incremented each time `processTurn()` triggers a burnout.
- Leaderboard: Persistent Map accumulates points across all races. 1st = 5 pts, 2nd = 3 pts, 3rd = 1 pt.

### Reward System — Leaderboard & Ranking (Option A)

- Points Algorithm: 5 / 3 / 1 for positions 1 / 2 / 3; 0 for all other places.
- Leaderboard Display: JTable sorted by cumulative points, refreshed after every race.
- Entries persist in memory for the session; re-launching the application resets them.

### TrackCanvas — Custom Race Visualisation

The race track is drawn entirely in `paintComponent()` using `Graphics2D`. Each typist is represented as a coloured circle (token) that moves along a horizontal lane. The token position is calculated as:  $trackX + (progress / passageLength) * trackWidth$ . A thin stamina bar above each token, a finish-line marker, lane dividers, and a burnt-out flash label complete the display. Anti-aliasing is enabled for smooth rendering.

## PART III — Git Integration

### Repository Structure

```
TypingRaceSimulator/  
■■■ https://github.com/TalhaKepenekci/Typing-Race-Simulator      # Git repository  
■■■ Part1/  
■   ■■■ Typist.java  
■   ■■■ TypingRace.java  
■■■ Part2/  
■   ■■■ TypingRaceGUI.java ■■■ README.md
```

### Git Workflow

| Step    | Command / Action                    | Purpose                                       |
|---------|-------------------------------------|-----------------------------------------------|
| Init    | git init                            | Initialise local repo in TypingRaceSimulator/ |
| Remote  | git remote add origin <url>         | Link to private GitHub repository             |
| Branch  | git checkout -b gui-development     | All Part II work on this branch               |
| Commits | git commit -m 'descriptive message' | Frequent, meaningful commits                  |
| Merge   | git merge gui-development           | Merged back to main after Part II complete    |
| Push    | git push origin main                | All history pushed to GitHub                  |

### Sample Commit Messages

- Add Typist class with all required fields and constructor
- Implement slideBack() with zero-floor clamping
- Fix win condition in TypingRace — changed == to >=
- Add TrackCanvas with per-lane token rendering
- Implement leaderboard with cumulative points across races
- Merge gui-development into main — all conflicts resolved

### README.md Contents

The README includes: project overview, prerequisites (JDK 11+), compilation commands for both parts, run commands, folder structure, and a brief description of each difficulty modifier and typist customisation option.

End of Report